EVIDENCIAS DE EJECUCIÓN E INTEGRACIÓN DEL PIPELINE

Semana 4 — Integrar y estabilizar el pipeline end-to-end

Proyecto	RapidGo 360 — Cadena de frío (variante 3)
Equipo	7
Responsable	Wilmer León
Fecha de ejecución	11 de noviembre de 2026

Objetivo

Documentar la ejecución local y la integración de los componentes del pipeline de cadena de frío, desde la generación de eventos hasta su procesamiento, almacenamiento en el Data Lake y evaluación mediante Airflow.

1. Alcance de la validación

La ejecución se realizó localmente desde la carpeta del repositorio descargado, utilizando Docker Compose y PowerShell. Se verificó el inicio de los servicios, la publicación de eventos, el procesamiento con Spark Structured Streaming, la escritura de datos en el Data Lake y la ejecución del DAG de calidad y KPI en Airflow.

2. Componentes integrados

- Zookeeper y Kafka: coordinación del broker y recepción de eventos en el tópico eventos-cadena-frio.
- Productor Python: generación de eventos JSON con identificadores únicos y un pedido mantenido durante la ejecución.
- Spark Structured Streaming: lectura desde Kafka, validación, transformación, cálculo de KPI y generación de alertas.
- Data Lake: persistencia de las capas raw, processed y curated, además de los reportes.
- Airflow: ejecución del DAG cadena_frio_calidad_reportes para verificar los datos, calcular indicadores y generar el reporte.

3. Secuencia de ejecución

3.1 Inicio de Kafka y Zookeeper

Se validó la configuración de Docker Compose y se iniciaron los servicios Kafka y Zookeeper. La salida de docker compose ps mostró ambos contenedores en estado activo.

```
PS C:\Users\wl244> cd C:\Users\wl244\Downloads\rapidgo-variant-3-cadena-frio-feature-s4-integracion-end-to-end
PS C:\Users\wl244\Downloads\rapidgo-variant-3-cadena-frio-feature-s4-integracion-end-to-end> docker compose config -q
PS C:\Users\wl244\Downloads\rapidgo-variant-3-cadena-frio-feature-s4-integracion-end-to-end> docker compose up -d zookee
per kafka
[+] up 17/18
 ⠿ Image confluentinc/cp-kafka:7.5.0                                                    Pulled                  225.6s
 ⠿ Image confluentinc/cp-zookeeper:7.5.0                                                Pulled                  225.6s
 ⠿ Network rapidgo-variant-3-cadena-frio-feature-s4-integracion-end-to-end_default       Created                   0.1s
 ⠿ Container rapidgo-variant-3-cadena-frio-feature-s4-integracion-end-to-end-zookeeper-1 Started                   1.2s
 ⠿ Container rapidgo-variant-3-cadena-frio-feature-s4-integracion-end-to-end-kafka-1     Started                   1.1s
PS C:\Users\wl244\Downloads\rapidgo-variant-3-cadena-frio-feature-s4-integracion-end-to-end> docker compose ps
NAME                                                                                   IMAGE                                COMMAND
                SERVICE     CREATED         STATUS         PORTS
rapidgo-variant-3-cadena-frio-feature-s4-integracion-end-to-end-kafka-1       confluentinc/cp-kafka:7.5.0       "/etc/co
nfluent/dock…"   kafka       6 seconds ago   Up 5 seconds   0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp
rapidgo-variant-3-cadena-frio-feature-s4-integracion-end-to-end-zookeeper-1   confluentinc/cp-zookeeper:7.5.0   "/etc/co
nfluent/dock…"   zookeeper   6 seconds ago   Up 5 seconds   2181/tcp, 2888/tcp, 3888/tcp
PS C:\Users\wl244\Downloads\rapidgo-variant-3-cadena-frio-feature-s4-integracion-end-to-end>
```

Figura 1. Inicio de Kafka y Zookeeper y verificación de contenedores.

3.2 Inicio de Spark Structured Streaming

Se inició el servicio spark-job. Los registros muestran la inicialización de Spark, la carga del conector de Kafka y la creación de las consultas de streaming. Los mensajes WARN visibles corresponden a advertencias de configuración; por sí solos no indican que la ejecución haya fallado.

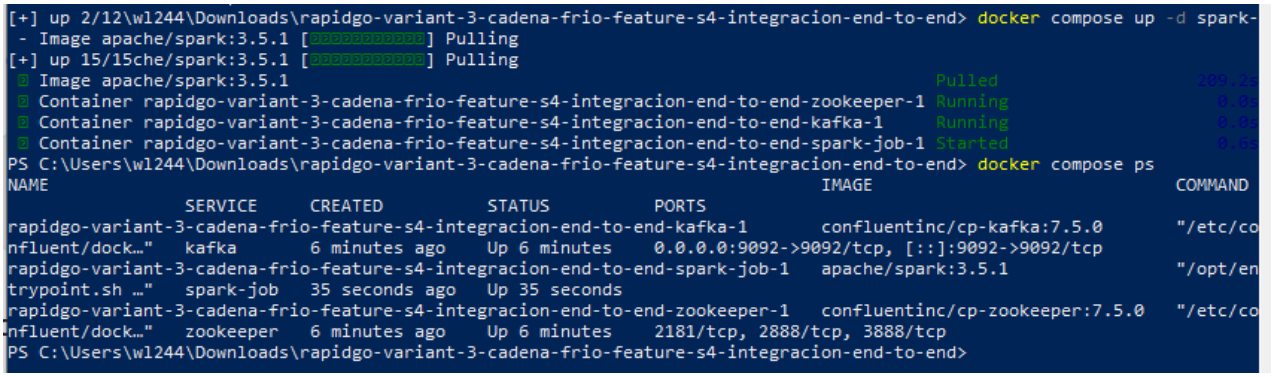


Figura 2. Inicio del servicio Spark y estado de los contenedores.

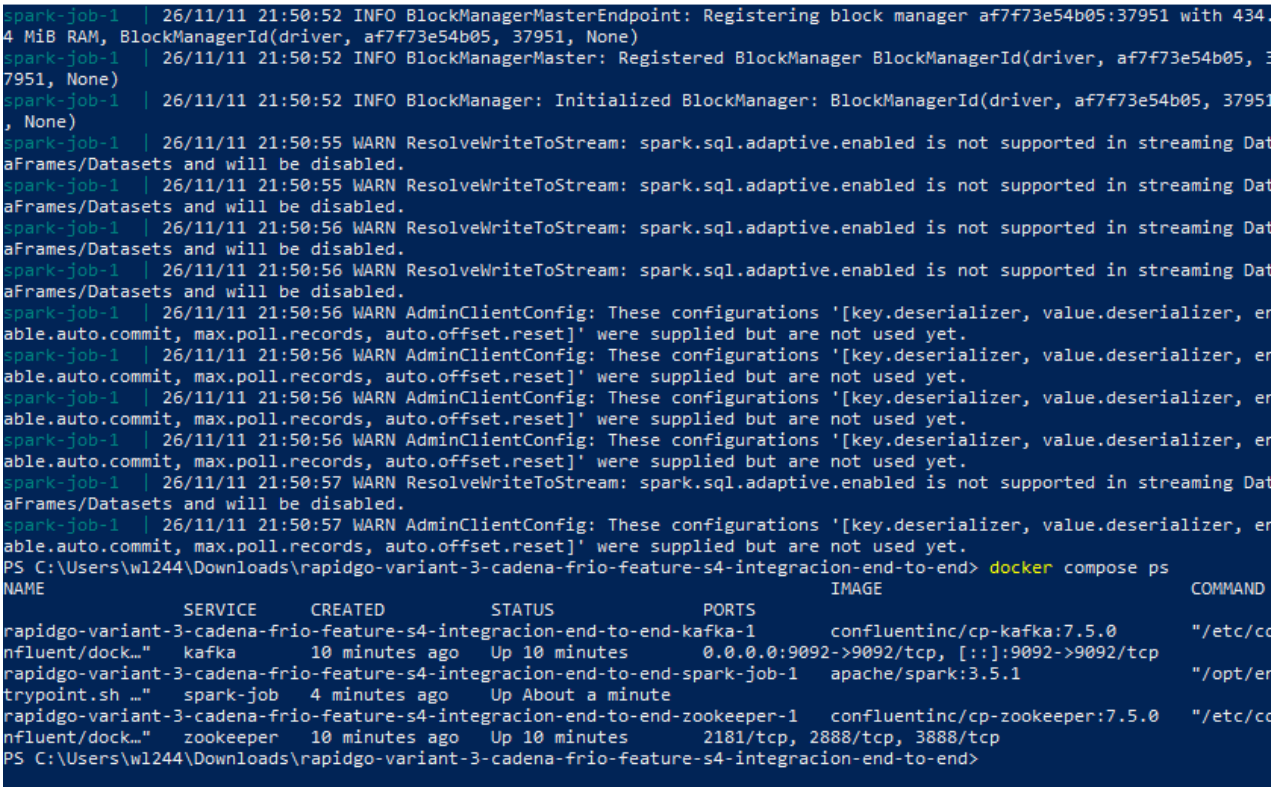


Figura 3. Registros de inicialización y actividad de Spark Structured Streaming.

3.3 Preparación y ejecución del productor

Se preparó un entorno Python dentro de un contenedor temporal, se instaló kafka-python y se ejecutó el productor. El productor alterna eventos normales y eventos de alerta, asigna un UUID a evento_id y conserva el mismo pedido_id durante la ejecución, permitiendo observar múltiples mediciones asociadas a un mismo pedido.

```
PS C:\Users\wl244\Downloads\rapidgo-variant-3-cadena-frio-feature-s4-integracion-end-to-end> docker run --rm -it --netwo
rk rapidgo-variant-3-cadena-frio-feature-s4-integracion-end-to-end_default -v "${PWD}\producer:/app/producer" -v "${PWD}
\samples:/app/samples" -w /app/producer python:3.12-slim bash
Unable to find image 'python:3.12-slim' locally
3.12-slim: Pulling from library/python
05e0eee5572e: Pull complete
68bced9e0afa: Pull complete
37fb86c2a767: Pull complete
2dc99d2d0a39: Pull complete
e0a5f9afcd02: Download complete
f26199dfe4a6: Download complete
Digest: sha256:5e2bdb1b0d542d5039cffb8181c65930093949194a65c365dfb1b0fe13a4fa5b
Status: Downloaded newer image for python:3.12-slim
root@2a688c47de60:/app/producer# pip install kafka-python
Collecting kafka-python
  Downloading kafka_python-3.0.13-py3-none-any.whl.metadata (11 kB)
Downloading kafka_python-3.0.13-py3-none-any.whl (614 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 614.3/614.3 kB 2.8 MB/s eta 0:00:00
Installing collected packages: kafka-python
Successfully installed kafka-python-3.0.13
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system packa
ge manager, possibly rendering your system unusable. It is recommended to use a virtual environment instead: https://pip
.pypa.io/warnings/venv. Use the --root-user-action option if you know what you are doing and want to suppress this warni
ng.

[notice] A new release of pip is available: 25.0.1 -> 26.3
[notice] To update, run: pip install --upgrade pip
root@2a688c47de60:/app/producer# sed -i "s/localhost:9092/kafka:29092/" producer.py
root@2a688c47de60:/app/producer# grep KAFKA_BROKER producer.py
KAFKA_BROKER = 'kafka:29092'
        bootstrap_servers=[KAFKA_BROKER],
root@2a688c47de60:/app/producer#
```

Figura 4. Preparación del entorno Python e instalación de kafka-python.

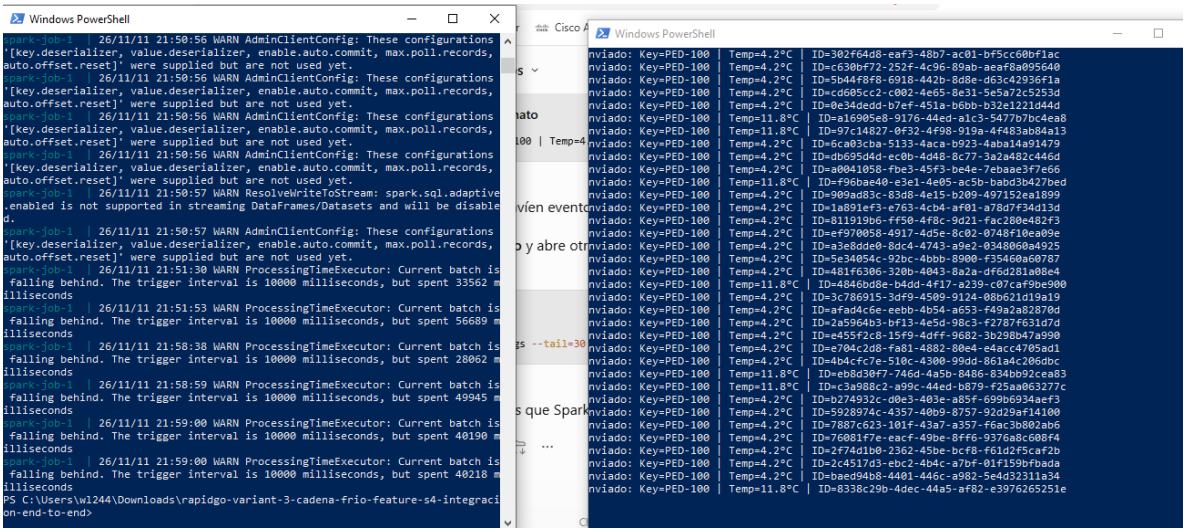


Figura 5. Productor enviando eventos y Spark procesando el flujo.

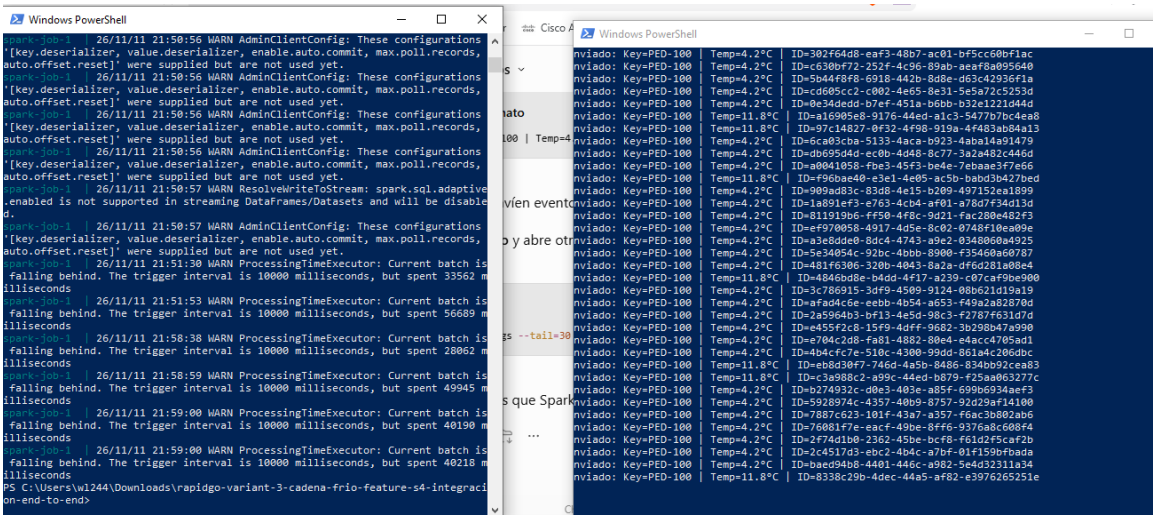


Figura 6. Estructura de directorios generada en el Data Lake.

3.4 Inicio y acceso a Airflow

Se inició Airflow mediante Docker Compose y se accedió a la interfaz web desde http://localhost:8081. La interfaz permitió visualizar el DAG cadena_frio_calidad_reportes.

```
PS C:\Users\wl244\Downloads\rapidgo-variant-3-cadena-frio-feature-s4-integracion-en
d-to-end> Get-ChildItem .\datalake -Directory


    Directorio: C:\Users\wl244\Downloads\rapidgo-variant-3-cadena-frio-feature-s4-
    integracion-end-to-end\datalake


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
da----      11/11/2026  4:49 p. m.                checkpoints
da----      11/11/2026  4:49 p. m.                curated
da----      11/11/2026  4:49 p. m.                processed
da----      11/11/2026  4:49 p. m.                raw


PS C:\Users\wl244\Downloads\rapidgo-variant-3-cadena-frio-feature-s4-integracion-en
d-to-end>
```

Figura 7. Inicio del servicio Airflow y verificación de su puerto.

```
PS C:\Users\wl244\Downloads\rapidgo-variant-3-cadena-frio-feature-s4-integracion-en
d-to-end> docker compose up -d airflow
[+] up 25/25
 ⠿ Image apache/airflow:2.9.3                                          Pulled  284.9s
 ⠿ Container rapidgo-variant-3-cadena-frio-feature-s4-integracion...   Started   3.5s
PS C:\Users\wl244\Downloads\rapidgo-variant-3-cadena-frio-feature-s4-integracion-en
d-to-end> docker compose ps
NAME                                                                          IMAGE
                          COMMAND                  SERVICE     CREATED
STATUS          PORTS
rapidgo-variant-3-cadena-frio-feature-s4-integracion-end-to-end-airflow-1     apach
e/airflow:2.9.3              "/usr/bin/dumb-init …"   airflow     18 seconds ago
Up 15 seconds   0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
rapidgo-variant-3-cadena-frio-feature-s4-integracion-end-to-end-kafka-1       confl
uentinc/cp-kafka:7.5.0       "/etc/confluent/dock…"   kafka       29 minutes ago
Up 29 minutes   0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp
rapidgo-variant-3-cadena-frio-feature-s4-integracion-end-to-end-spark-job-1   apach
e/spark:3.5.1                "/opt/entrypoint.sh …"   spark-job   23 minutes ago
Up 20 minutes
rapidgo-variant-3-cadena-frio-feature-s4-integracion-end-to-end-zookeeper-1   confl
uentinc/cp-zookeeper:7.5.0   "/etc/confluent/dock…"   zookeeper   29 minutes ago
Up 29 minutes   2181/tcp, 2888/tcp, 3888/tcp
PS C:\Users\wl244\Downloads\rapidgo-variant-3-cadena-frio-feature-s4-integracion-en
d-to-end>
```

Figura 8. Pantalla de acceso a Airflow.

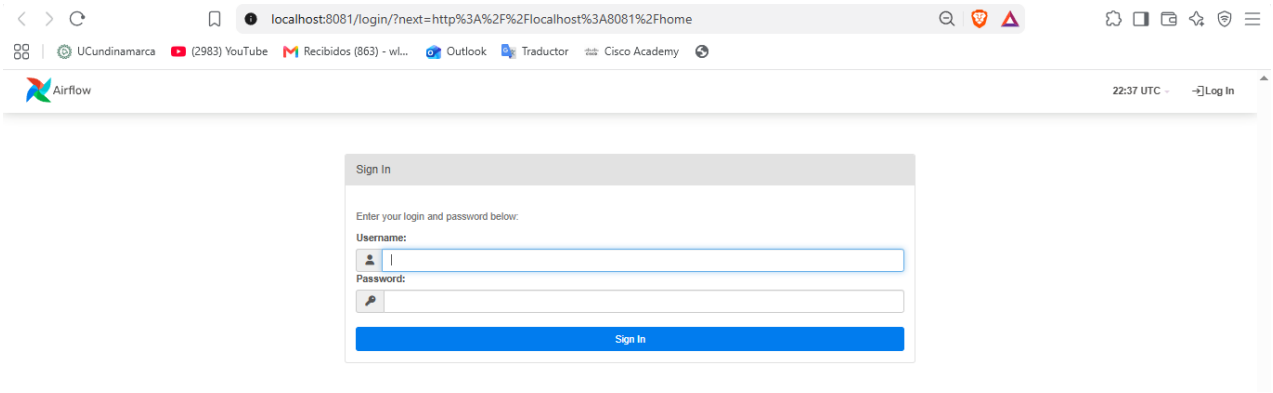


Figura 9. DAG cadena_frio_calidad_reportes visible en la interfaz.

3.5 Ejecución del DAG

Se activó el DAG y se ejecutaron sus seis tareas: verificar_datalake, validar_datos, calidad_datos, calcular_kpis, generar_reporte y resumen_kpi. La vista de ejecución mostró las tareas completadas correctamente y el resumen registró una ejecución exitosa.

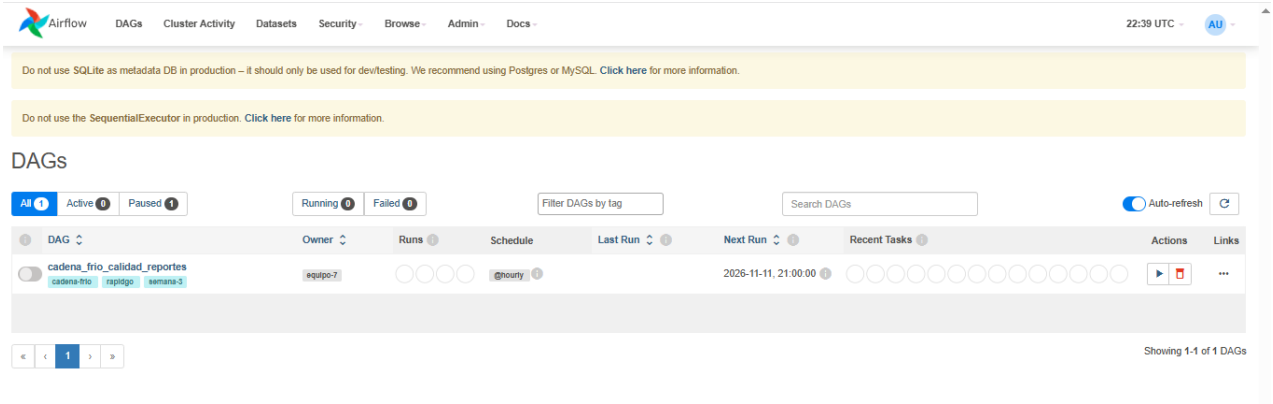


Figura 10. Vista de ejecución del DAG y resumen de tareas.

4. Resultados de la integración

El reporte generado por Airflow para el 11 de noviembre de 2026 indicó estado de calidad APROBADO. Los valores que siguen se transcriben del reporte de ejecución compartido.

Indicador	Resultado	Observación
Filas procesadas	208	Eventos incluidos en processed/eventos
Pedidos distintos	1	Se mantuvo un mismo pedido durante la prueba
Filas con nulos obligatorios	0	Sin nulos detectados
Valores físicamente imposibles	0	Sin valores fuera de límites físicos
Duplicados por partición/offset Kafka	0	Sin duplicados según la clave usada
Completitud	100,0 %	Umbral: 95,0 %
Validez	100,0 %	Umbral: 95,0 %
Unicidad	100,0 %	Umbral: 95,0 %
Retención raw → processed	100,0 %	208 → 208; umbral: 90,0 %
Minutos acumulados fuera de rango	1,24	KPI 1
Episodios fuera de rango	27	Duración máxima indicada: 0,13 min
Tiempo dentro de rango	82,23 %	KPI 3
Alertas de temperatura crítica	37	Eventos detectados
Alertas de exposición prolongada	0	No se detectaron en esta ejecución

El reporte también indicó 17 archivos Parquet en raw/eventos, 16 en processed/eventos, 14 en curated/kpis_ventana y 12 en curated/alertas_temperatura_critica. La carpeta curated/alertas_exposicion_prolongada no registró archivos en esa ejecución.

5. Archivos de reporte generados

Airflow generó los siguientes archivos dentro de datalake/reportes:

- reporte_cadena_frio_2026-11-11.json
- reporte_cadena_frio_2026-11-11.md
- reporte_cadena_frio_2026-11-11_pedidos.csv



Figura 11. Archivos JSON, Markdown y CSV generados en datalake/reportes.

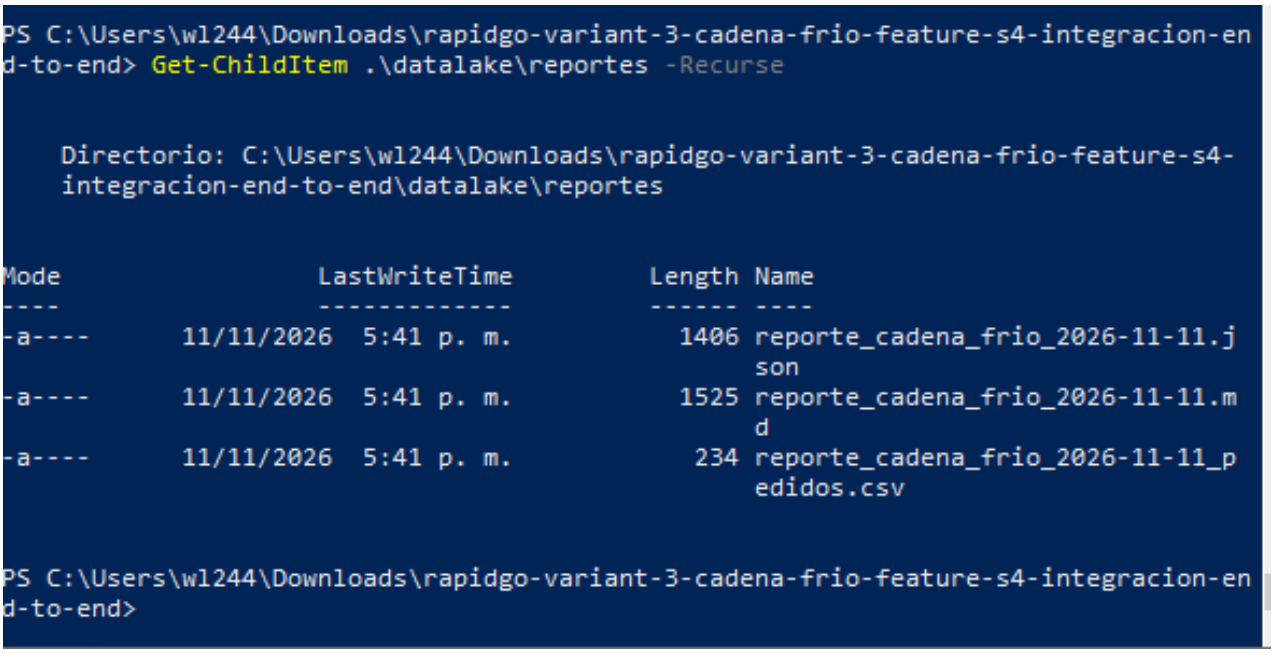


Figura 12. Evidencia adicional de la carpeta de reportes (captura original).

6. Observaciones y alcance de la prueba

- La ejecución demuestra el recorrido de eventos desde el productor hacia Kafka, su procesamiento mediante Spark y la persistencia de resultados en el Data Lake.
- El DAG de Airflow finalizó con estado exitoso y el reporte de calidad quedó aprobado.
- La ausencia de alertas de exposición prolongada significa que no se detectaron casos que cumplieran esa regla durante esta ejecución; no equivale a una prueba controlada de dicha condición.
- El reporte mostró un único pedido monitorizado, coherente con el ajuste del productor para conservar el mismo pedido durante la ejecución.
- Las advertencias de Spark observadas en los logs deben distinguirse de errores fatales: en las capturas compartidas el servicio permaneció activo y continuó generando datos.

7. Verificación final de CI y servicios

Las siguientes evidencias complementan la ejecución local y permiten cerrar la verificación:

- GitHub Actions: ejecución #50 finalizada correctamente (Success), con todas las etapas del workflow completadas.
- Docker Compose: los seis servicios —Airflow, Kafka, Spark Job, Spark Master, Spark Worker y Zookeeper— aparecen en estado Up.

Con estas dos capturas quedan documentados tanto el resultado satisfactorio de GitHub Actions como el estado activo de los servicios desplegados con Docker Compose.

8. Evidencias complementarias de cierre

8.1 Validación continua con GitHub Actions

La ejecución #50 del workflow de CI finalizó con estado exitoso. En el detalle se observan completadas las etapas de descarga del repositorio, configuración de Java 17 y Python, instalación de dependencias, validación de sintaxis Python, pruebas automatizadas, validación de Docker Compose y validación de sintaxis de los DAGs de Airflow.

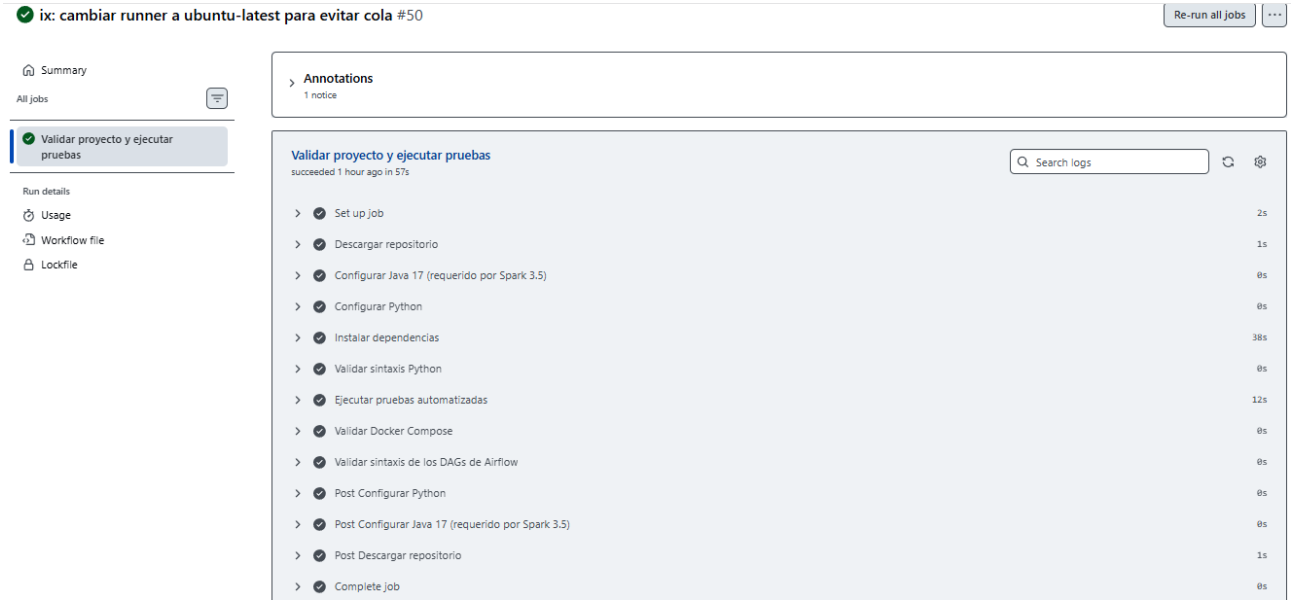


Figura 13. GitHub Actions — workflow #50 completado correctamente.

8.2 Estado final de los servicios Docker

La salida de docker compose ps confirma que los seis servicios del entorno se encuentran activos (Up): Airflow, Kafka, Spark Job, Spark Master, Spark Worker y Zookeeper. También se muestran los puertos publicados para acceder a Airflow, Kafka y Spark Master.

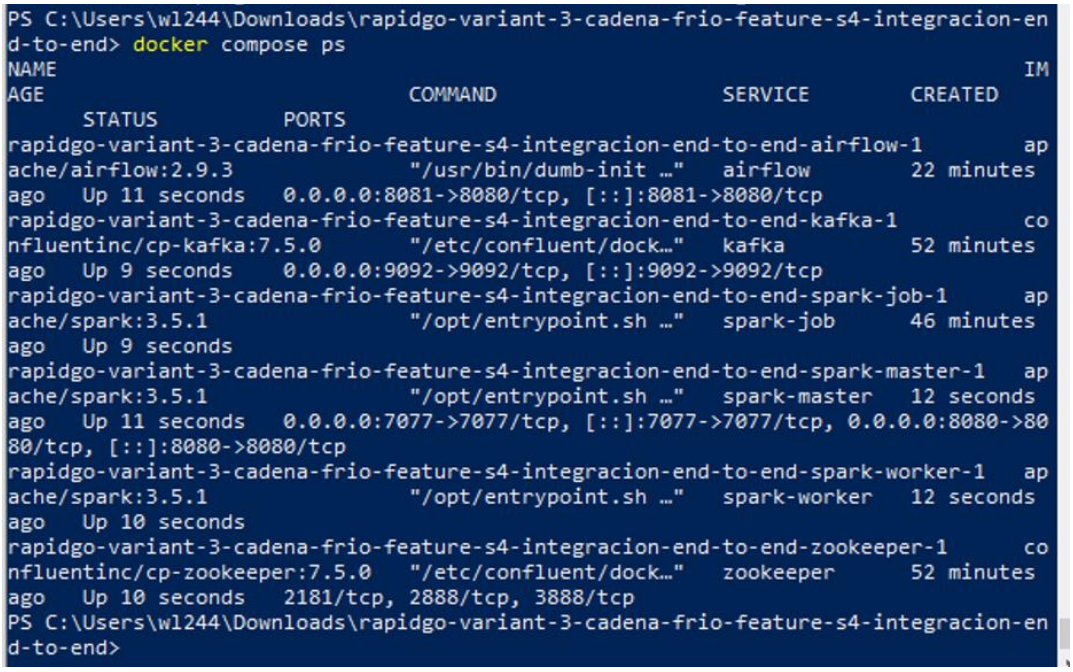


Figura 14. Estado de los seis servicios del pipeline mediante docker compose ps.

9. Cierre de la evidencia

Con las evidencias de ejecución de Kafka, Spark, el productor, el Data Lake y Airflow; el reporte de calidad aprobado; el workflow de CI exitoso; y los seis servicios Docker en estado activo, queda documentada la integración local solicitada para la Semana 4. La prueba controlada de exposición prolongada no se presenta como realizada, ya que no fue necesaria para demostrar el resultado de esta ejecución y el reporte registró cero alertas de ese tipo.

10. Conclusión

La integración local del pipeline de cadena de frío se ejecutó satisfactoriamente en el entorno documentado. Se verificó la disponibilidad de Kafka y Zookeeper, el procesamiento de eventos con Spark Structured Streaming, la escritura de datos en las distintas capas del Data Lake y la ejecución exitosa del DAG de Airflow. El reporte final obtuvo estado APROBADO y generó los archivos JSON, Markdown y CSV correspondientes. Las evidencias complementarias de CI y del estado final de todos los contenedores se incorporarán antes del cierre definitivo del Issue #11.